LAPORAN PRAKTIKUM

Struktur Data

MODUL 6

DOUBLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Elsa Dwi Rizqiyanti

109082500090

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 6 : Struktur-Data/Modul6 at main · elsadwiyt/Struktur-Data

1. **Doublylist.h**

Source Code

```
#ifndef DOUBLYLIST_H_INCLUDED
#define DOUBLYLIST_H_INCLUDED

#include <iostream>
#include <string>

using namespace std;

struct infotype {
    string nopol, warna;
    int thnBuat;
};

typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next, prev;
};

struct List {
    address First, Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
address findElm(List L, string nopol);
void insertLast(List &L, address P);
void printInfo(List L);
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address &Prec, address &P);
void deleteSpecific(List &L, string nopol);

#endif
```

Deskripsi program

Program Doublylist.h ini tuh kayak cetak biru atau kerangka dasar buat bikin Doubly Linked List khusus data kendaraan. Di dalamnya udah disiapin wadah buat nyimpen data, bentuk node-nya yang punya tali penghubung next dan prev, penanda ujung list, serta

daftar fungsi atau prosedur penting seperti CreateList, insertLast, findElm, sampai fungsi-fungsi delete yang nanti tinggal kita jabarkan detail kodenya di file .cpp.

2. Doublylist.cpp

Source Code

```
#include "Doublylist.h"

void CreateList(List &L) {
    L.First = NULL;
    L.Last = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

address findElm(List L, string nopol) {
    address P = L.First;
    while (P != NULL) {
        if (P->info.nopol == nopol) return P;
        P = P->next;
    }
    return NULL;
}

void insertLast(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        L.Last = P;
    } else {
        L.Last->next = P;
        P->prev = L.Last;
        L.Last = P;
    }
}

void printInfo(List L) {
    address P = L.Last;
    while (P != NULL) {
```

```
            cout << "no polisi : " << P->info.nopol << endl;
            cout << "warna     : " << P->info.warna << endl;
            cout << "tahun     : " << P->info.thnBuat << endl;
            P = P->prev;
        }
    }

    void deleteFirst(List &L, address &P) {
        if (L.First == NULL) {
            P = NULL;
        } else if (L.First == L.Last) {
            P = L.First;
            L.First = NULL;
            L.Last = NULL;
        } else {
            P = L.First;
            L.First = L.First->next;
            L.First->prev = NULL;
            P->next = NULL;
        }
    }

    void deleteLast(List &L, address &P) {
        if (L.First == NULL) {
            P = NULL;
        } else if (L.First == L.Last) {
            P = L.First;
            L.First = NULL;
            L.Last = NULL;
        } else {
            P = L.Last;
            L.Last = L.Last->prev;
            L.Last->next = NULL;
            P->prev = NULL;
        }
    }

    void deleteAfter(address &Prec, address &P) {
        if (Prec != NULL && Prec->next != NULL) {
            P = Prec->next;
            Prec->next = P->next;
            if (P->next != NULL) P->next->prev = Prec;
            P->next = NULL;
            P->prev = NULL;
        } else {
            P = NULL;
        }
    }
```

```
void deleteSpecific(List &L, string nopol) {
    address P = findElm(L, nopol);
    if (P != NULL) {
        if (P == L.First) deleteFirst(L, P);
        else if (P == L.Last) deleteLast(L, P);
        else {
            address Prec = P->prev;
            deleteAfter(Prec, P);
        }
        dealokasi(P);
        cout << "Data dengan nomor polisi " << nopol << "
berhasil dihapus.\n";
    } else {
        cout << "Data tidak ditemukan.\n";
    }
}
```

Deskripsi program

Program Doublylist.cpp ini adalah tempat kita menuliskan isi atau cara kerja dari semua fungsi dan prosedur yang kerangkanya sudah dibuat di file header sebelumnya, mulai dari bikin list kosong, nyiapin kotak data baru, nyari nomor polisi, nambahin data di ujung, nampilin data dari belakang ke depan, sampai berbagai aturan cara ngapus data (deleteFirst, deleteLast, deleteAfter, dan deleteSpecific).

3. **Main.cpp**

Source Code

```
#include "Doublylist.h"

int main() {
    List L;
    CreateList(L);
    infotype x;
    address P;

    for (int i = 0; i < 4; i++) {
        cout << "masukkan nomor polisi: ";
        cin >> x.nopol;
        cout << "masukkan warna kendaraan: ";
        cin >> x.warna;
        cout << "masukkan tahun kendaraan: ";
        cin >> x.thnBuat;

        if (findElm(L, x.nopol) != NULL) {
            cout << "nomor polisi sudah terdaftar\n";
        } else {
            P = alokasi(x);
```

```
                insertLast(L, P);
            }
        }

        cout << "\n==================================\n";

        string cariNopol;
        cout << "Masukkan Nomor Polisi yang dicari : ";
        cin >> cariNopol;

        address hasilCari = findElm(L, cariNopol);
        if (hasilCari != NULL) {
            cout << "Nomor Polisi : " << hasilCari->info.nopol <<
    endl;
            cout << "Warna        : " << hasilCari->info.warna <<
    endl;
            cout << "Tahun        : " << hasilCari->info.thnBuat
    << endl;
        } else {
            cout << "Nomor polisi tidak ditemukan.\n";
        }

        cout << "\n==================================\n";

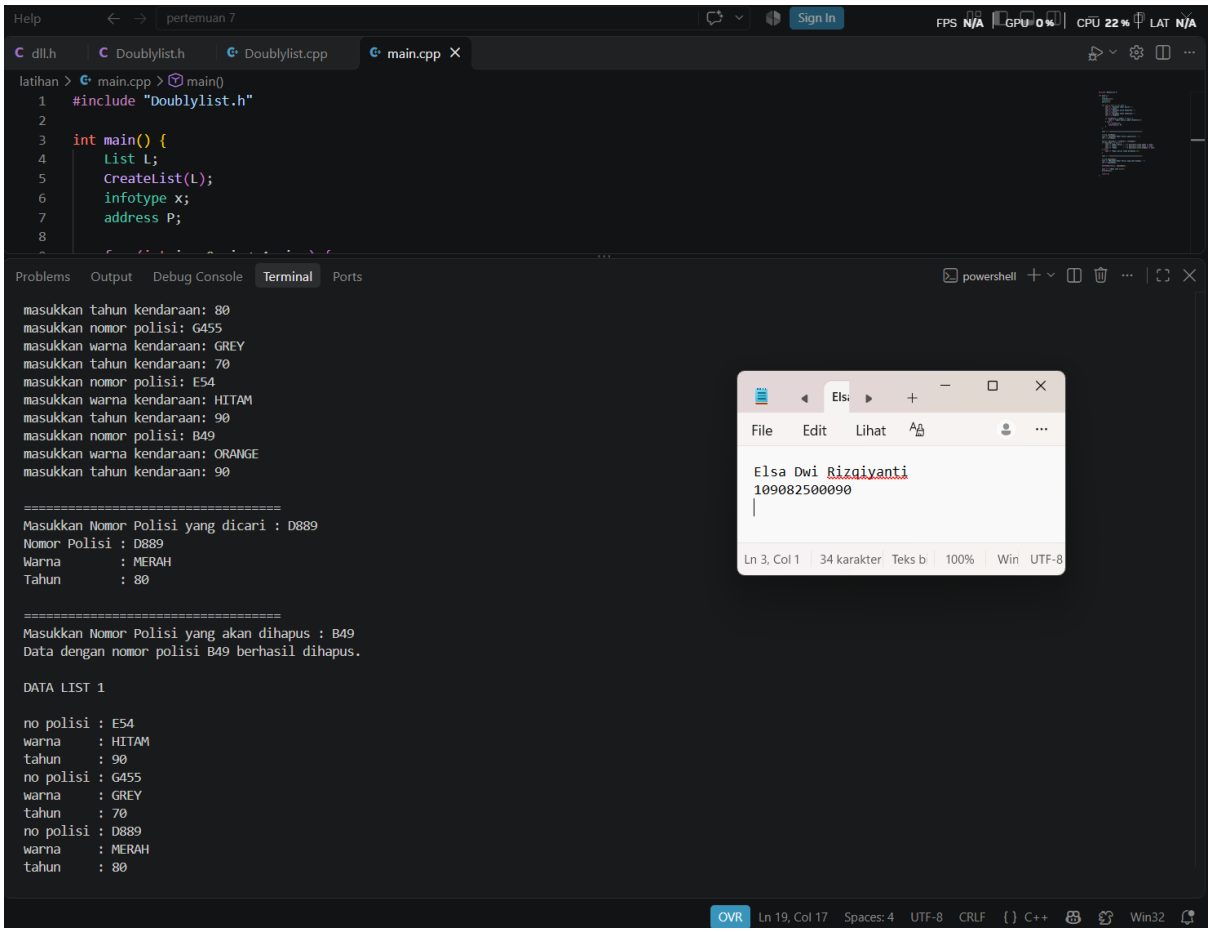
        string hapusNopol;
        cout << "Masukkan Nomor Polisi yang akan dihapus : ";
        cin >> hapusNopol;

        deleteSpecific(L, hapusNopol);

        cout << "\nDATA LIST 1\n\n";
        printInfo(L);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program main.cpp ini adalah panggung utamanya tempat program dijalankan secara berurutan: pertama kita diminta menginput data 4 kendaraan (sambil ngecek biar nopolnya nggak kembar), terus lanjut ngejalanin fitur buat nyari data kendaraan pakai findElm, ngapus data kendaraan tertentu pakai deleteSpecific, dan terakhir nampilin sisa data kendaraan yang ada di dalam *list* dari belakang ke depan lewat printInfo.

4. **dll.h (guided)**

Source Code

```
#ifndef DLL_H
#define DLL_H

#include <iostream>
#include <string>
using namespace std;

struct Kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};
struct ElmList {
    Kendaraan info;
    ElmList* next;
    ElmList* prev;
};

// STRUKTUR LIST
struct List {
    ElmList* first;
    ElmList* last;
};

// FUNGSI DASAR
void createList(List &L);
ElmList* alokasi(Kendaraan x);
void dealokasi(ElmList* &P);
void printInfo(List L);
ElmList* findElm(List L, string nopol);

// FUNGSI INSERT
void insertFirst(List &L, ElmList* P);
void insertLast(List &L, ElmList* P);
void insertAfter(List &L, ElmList* Prec, ElmList* P);
void insertBefore(List &L, ElmList* Succ, ElmList* P);

// FUNGSI DELETE
void deleteFirst(List &L, ElmList* &P);
void deleteLast(List &L, ElmList* &P);
void deleteAfter(List &L, ElmList* Prec, ElmList* &P);
void deleteBefore(List &L, ElmList* Succ, ElmList* &P);

#endif
```

Deskripsi program

Program dll.h ini mirip seperti sebelumnya yaitu berkas header kerangka dasar untuk Doubly Linked List data kendaraan tetapi dengan sedikit variasi nama struktur serta kelengkapan fungsi tambahan yang lebih lengkap seperti insertFirst, insertBefore, deleteBefore, dan deleteAfter yang siap diimplementasikan kodenya di file.cpp.

5. dll.cpp (guided)

Source Code

```
#include "dll.h"

// MEMBUAT LIST KOSONG
void createList(List &L) {
    L.first = nullptr;
    L.last = nullptr;
}

// MEMBUAT NODE BARU
ElmList* alokasi(Kendaraan x) {
    ElmList* P = new ElmList;

    P->info = x;
    P->next = nullptr;
    P->prev = nullptr;

    return P;
}

// MEMBEBASKAN MEMORI NODE
void dealokasi(ElmList* &P) {
    if (P != nullptr) {
        delete P;
        P = nullptr;
    }
}

// MENAMPILKAN SEMUA DATA KENDARAAN
void printInfo(List L) {
    ElmList* P = L.first;

    if (P == nullptr) {
        cout << "List kosong." << endl;
        return;
    }

    while (P != nullptr) {
```

```
            cout << "Nopol    : " << P->info.nopol << endl;
            cout << "Warna    : " << P->info.warna << endl;
            cout << "Tahun    : " << P->info.thnBuat << endl;
            cout << "-------------------" << endl;

            P = P->next;
        }
    }

    // MENCARI NODE BERDASARKAN NOMOR POLISI
    ElmList* findElm(List L, string nopol) {
        ElmList* P = L.first;

        while (P != nullptr) {
            if (P->info.nopol == nopol) {
                return P;
            }

            P = P->next;
        }

        return nullptr;
    }

    // INSERT FIRST: MENAMBAHKAN NODE DI AWAL
    void insertFirst(List &L, ElmList* P) {
        if (P == nullptr) {
            return;
        }

        P->prev = nullptr;
        P->next = L.first;

        if (L.first != nullptr) {
            L.first->prev = P;
        } else {
            L.last = P;
        }

        L.first = P;
    }

    // INSERT LAST: MENAMBAHKAN NODE DI AKHIR
    void insertLast(List &L, ElmList* P) {
        if (P == nullptr) {
            return;
        }
```

```
    P->next = nullptr;
    P->prev = L.last;

    if (L.last != nullptr) {
        L.last->next = P;
    } else {
        L.first = P;
    }

    L.last = P;
}

// INSERT AFTER: MENAMBAHKAN NODE SETELAH NODE ACUAN
void insertAfter(List &L, ElmList* Prec, ElmList* P) {
    if (Prec == nullptr || P == nullptr) {
        return;
    }

    if (Prec == L.last) {
        insertLast(L, P);
        return;
    }

    P->next = Prec->next;
    P->prev = Prec;

    Prec->next->prev = P;
    Prec->next = P;
}

// INSERT BEFORE: MENAMBAHKAN NODE SEBELUM NODE ACUAN
void insertBefore(List &L, ElmList* Succ, ElmList* P) {
    if (Succ == nullptr || P == nullptr) {
        return;
    }

    if (Succ == L.first) {
        insertFirst(L, P);
        return;
    }

    P->next = Succ;
    P->prev = Succ->prev;

    Succ->prev->next = P;
    Succ->prev = P;
}
```

```
// DELETE FIRST: MENGELUARKAN NODE PERTAMA
void deleteFirst(List &L, ElmList* &P) {
    if (L.first == nullptr) {
        P = nullptr;
        return;
    }

    P = L.first;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.first = P->next;
        L.first->prev = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE LAST: MENGELUARKAN NODE TERAKHIR
void deleteLast(List &L, ElmList* &P) {
    if (L.last == nullptr) {
        P = nullptr;
        return;
    }

    P = L.last;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.last = P->prev;
        L.last->next = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE AFTER: MENGELUARKAN NODE SETELAH NODE ACUAN
void deleteAfter(List &L, ElmList* Prec, ElmList* &P) {
    if (Prec == nullptr || Prec->next == nullptr) {
        P = nullptr;
        return;
    }
```

```
    P = Prec->next;

    if (P == L.last) {
        deleteLast(L, P);
        return;
    }

    Prec->next = P->next;
    P->next->prev = Prec;

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE BEFORE: MENGELUARKAN NODE SEBELUM NODE ACUAN
void deleteBefore(List &L, ElmList* Succ, ElmList* &P) {
    if (Succ == nullptr || Succ->prev == nullptr) {
        P = nullptr;
        return;
    }

    P = Succ->prev;

    if (P == L.first) {
        deleteFirst(L, P);
        return;
    }

    Succ->prev = P->prev;
    P->prev->next = Succ;

    P->next = nullptr;
    P->prev = nullptr;
}
```

Deskripsi program

Program implementasi ini berisi kumpulan kode lengkap yang menjalankan seluruh fungsi dan prosedur Doubly Linked List untuk data kendaraan mulai dari inisialisasi list kosong, alokasi dan dealokasi memori, pencarian data, pencetakan data, hingga berbagai variasi operasi penambahan dan penghapusan elemen yang lengkap baik di awal, di akhir, maupun di sekitar node acuan tertentu (insertFirst, insertLast, insertAfter, insertBefore, deleteFirst, deleteLast, deleteAfter, dan deleteBefore).

6. **Main.cpp (guided)**

Source Code

```
#include "dll.h"

int main() {
    List L;
    createList(L);

    Kendaraan K1 = {"D001", "Merah", 2022};
    Kendaraan K2 = {"D002", "Hitam", 2023};
    Kendaraan K3 = {"D003", "Biru", 2024};
    Kendaraan K4 = {"D004", "Abu-abu", 2025};
    Kendaraan K0 = {"D000", "Putih", 2021};

    // INSERT LAST: MEMASUKKAN D001 DI AKHIR
    ElmList* P = alokasi(K1);
    insertLast(L, P);

    // INSERT LAST: MEMASUKKAN D002 DI AKHIR
    P = alokasi(K2);
    insertLast(L, P);

    // INSERT FIRST: MEMASUKKAN D000 DI AWAL
    P = alokasi(K0);
    insertFirst(L, P);

    // INSERT AFTER: MEMASUKKAN D003 SETELAH D001
    ElmList* Prec = findElm(L, "D001");

    P = alokasi(K3);
    insertAfter(L, Prec, P);

    // INSERT BEFORE: MEMASUKKAN D004 SEBELUM D002
    ElmList* Succ = findElm(L, "D002");

    P = alokasi(K4);
    insertBefore(L, Succ, P);

    // MENAMPILKAN LIST SETELAH INSERT
    cout << "=== DATA SETELAH INSERT ===" << endl;
    printInfo(L);

    // FIND ELEMENT: MENCARI D001
    P = findElm(L, "D001");

    cout << "=== HASIL PENCARIAN ===" << endl;
```

```
        if (P != nullptr) {
            cout << "Nopol ditemukan: " << P->info.nopol << endl;
            cout << "Warna: " << P->info.warna << endl;
            cout << "Tahun: " << P->info.thnBuat << endl;
        } else {
            cout << "Kendaraan tidak ditemukan." << endl;
        }

        // DELETE AFTER: MENGHAPUS D003 SETELAH D001
        Prec = findElm(L, "D001");

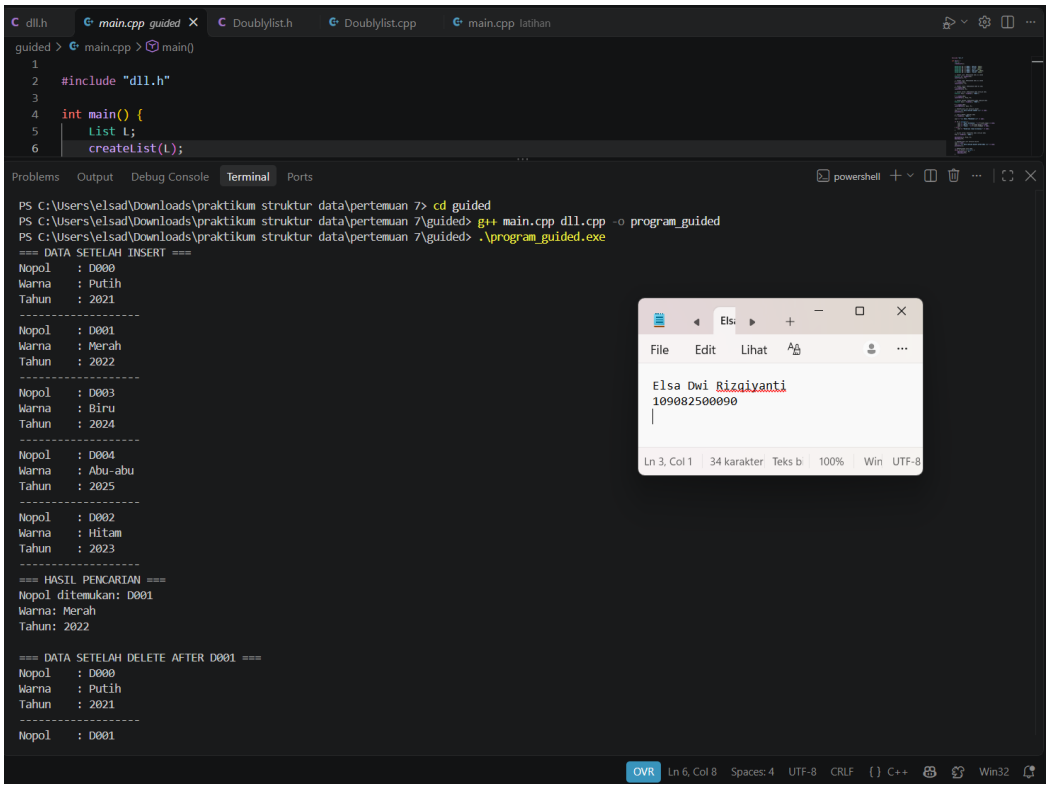
        deleteAfter(L, Prec, P);
        dealokasi(P);

        // MENAMPILKAN LIST SETELAH DELETE
        cout << endl;
        cout << "=== DATA SETELAH DELETE AFTER D001 ===" << endl;
        printInfo(L);

        // MEMBERSIHKAN SISA NODE
        while (L.first != nullptr) {
            deleteFirst(L, P);
            dealokasi(P);
        }

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program main.cpp ini adalah program utama untuk menguji seluruh fungsi operasi Doubly Linked List kendaraan mulai dari memasukkan data menggunakan insertLast, insertFirst, insertAfter, dan insertBefore, menampilkan data, mencari nomor polisi tertentu, menghapus node menggunakan deleteAfter, hingga membersihkan seluruh sisa memori list di akhir program.